Qwen2.5-1.5B-Instruct — end-to-end architecture

1.54 B parameters = token embedding 233.4 M (shared with the LM head) + 28 decoder layers 1.31 B. Text in, text out: no vision part.

Shapes [rows, features]: L = prompt length in tokens,
d = 1536 (model width), V = 151,936 (vocab rows)
Example in all figures: “What is 2+2?” → L = 36
answer “2 + 2 equals 4.” = 9 tokens incl. <|im_end|>

PROMPT → TOKEN IDS (CPU, before the model)

Prompt
one user
message:
“What is 2+2?”

→

Chat template
<|im_start|>system
You are Qwen, created by Alibaba Cloud.
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is 2+2?<|im_end|>
<|im_start|>assistant

text →

Tokenizer
byte-level BPE
151,643 tokens
+ 22 special (<|im_start|> ...)
= 151,665 ids

[36] ids →

Token embedding
lookup table V × d
= 151,936 × 1536
one row per token id
rows ≥ 151,665: unused padding

[36,1536] →

The 36 prompt tokens of the example (␣ = space, \n = newline); position under every 5th:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
<|im_start|>	system	\n	You	␣are	␣Q	wen	,	␣created	␣by	␣Alibaba	␣Cloud	.	␣You	␣are	␣a	␣helpful	␣assistant	.	<|im_end|>	\n	<|im_start|>	user	\n	What	␣is	␣	2	+	2	?	<|im_end|>	\n	<|im_start|>	assistant	\n

LANGUAGE MODEL

Positions (RoPE)
position ids 0 … L−1
→ cos / sin, used to
rotate q, k in every layer

Decoder layer ×28
pre-RMSNorm + residual
self-attention (GQA, causal)
12 query heads, 2 KV heads
head dim 128, RoPE on q, k
SwiGLU MLP 1536→8960→1536
KV cache for decoding

[36, 1536]

Final RMSNorm

[1, 1536] last pos.

LM head
Linear 1536 → 151,936
no bias
same matrix as the embedding

[1, 151,936] logits

Next token
argmax (greedy, as in
the benchmark) → “2”
only the last position

DECODE LOOP — one token per pass, earlier keys / values come from the KV cache

pass	prefill	decode 1	decode 2	decode 3	decode 4	decode 5	decode 6	decode 7	decode 8
token out	2	␣+	␣	2	␣equals	␣	4	.	<|im_end|>
	36 in	1 in	1 in	1 in	1 in	1 in	1 in	1 in	1 in
KV cache	0 → 36	37 → 38	38 → 39	39 → 40	40 → 41	41 → 42	42 → 43	43 → 44	44 → 45

The pass that reads the 36-token prompt also produces the first token (time to first token). <|im_end|> ends the turn: 9 passes in all.

Sizes of this model (from config.json)

```
width d = 1536, 28 layers
12 q-heads / 2 KV-heads × 128
MLP 1536→8960→1536
vocab rows 151,936, tied head: yes
RoPE θ = 1,000,000, RMSNorm ε = 1e-06
```

Where the parameters are



- token embedding = LM head: 233.4 M (15.1 %)
- attention ×28: 154.2 M (10.0 %)
- MLP ×28: 1.16 B (74.9 %)
- RMSNorms: 0.1 M (0.0 %)

bf16 weights: 3.09 GB · KV cache: 28 KiB per token · ≈ 3.1 GFLOPs per token (linear layers)

Qwen2.5-3B-Instruct — end-to-end architecture

3.09 B parameters = token embedding 311.2 M (shared with the LM head) + 36 decoder layers 2.77 B. Text in, text out: no vision part.

Shapes [rows, features]: L = prompt length in tokens,
d = 2048 (model width), V = 151,936 (vocab rows)
Example in all figures: “What is 2+2?” → L = 36
answer “2 + 2 equals 4.” = 9 tokens incl. <|im_end|>

PROMPT → TOKEN IDS (CPU, before the model)

Prompt
one user
message:
“What is 2+2?”

Chat template
<|im_start|>system
You are Qwen, created by Alibaba Cloud.
You are a helpful assistant.<|im_end|>
<|im_start|>user
What is 2+2?<|im_end|>
<|im_start|>assistant

text

Tokenizer
byte-level BPE
151,643 tokens
+ 22 special (<|im_start|> ...)
= 151,665 ids

[36] ids

Token embedding
lookup table V × d
= 151,936 × 2048
one row per token id
rows ≥ 151,665: unused padding

[36,2048]

The 36 prompt tokens of the example (␣ = space, \n = newline); position under every 5th:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
<|im_start|>	system	\n	You	␣are	␣Q	wen	,	␣created	␣by	␣Alibaba	␣Cloud	.	␣You	␣are	␣a	␣helpful	␣assistant	.	<|im_end|>	\n	<|im_start|>	user	\n	What	␣is	␣	2	+	2	?	<|im_end|>	\n	<|im_start|>	assistant	\n

LANGUAGE MODEL

Positions (RoPE)
position ids 0 … L−1
→ cos / sin, used to
rotate q, k in every layer

Decoder layer ×36
pre-RMSNorm + residual
self-attention (GQA, causal)
16 query heads, 2 KV heads
head dim 128, RoPE on q, k
SwiGLU MLP 2048→11008→2048
KV cache for decoding

[36, 2048]

Final RMSNorm

[1, 2048] last pos.

LM head
Linear 2048 → 151,936
no bias
same matrix as the embedding

[1, 151,936] logits

Next token
argmax (greedy, as in
the benchmark) → “2”
only the last position

DECODE LOOP — one token per pass, earlier keys / values come from the KV cache

pass	prefill	decode 1	decode 2	decode 3	decode 4	decode 5	decode 6	decode 7	decode 8
token out	2	␣+	␣	2	␣equals	␣	4	.	<|im_end|>
	36 in	1 in	1 in	1 in	1 in	1 in	1 in	1 in	1 in
KV cache	0 → 36	37 → 38	38 → 39	39 → 40	40 → 41	41 → 42	42 → 43	43 → 44	44 → 45

The pass that reads the 36-token prompt also produces the first token (time to first token). <|im_end|> ends the turn: 9 passes in all.

Sizes of this model (from config.json)

width d = 2048, 36 layers
16 q-heads / 2 KV-heads × 128
MLP 2048→11008→2048
vocab rows 151,936, tied head: yes
RoPE θ = 1,000,000, RMSNorm ε = 1e-06

Where the parameters are



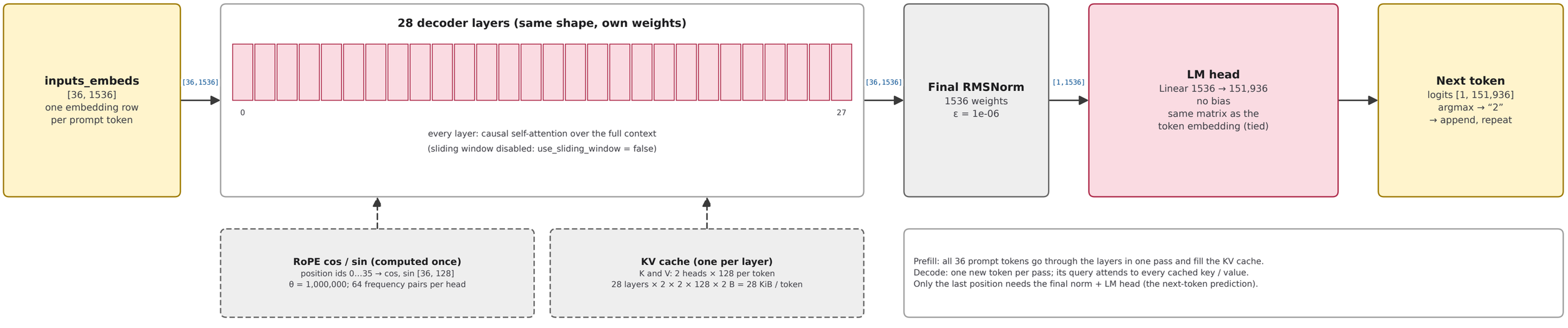
- token embedding = LM head: 311.2 M (10.1 %)
- attention ×36: 339.8 M (11.0 %)
- MLP ×36: 2.43 B (78.9 %)
- RMSNorms: 0.1 M (0.0 %)

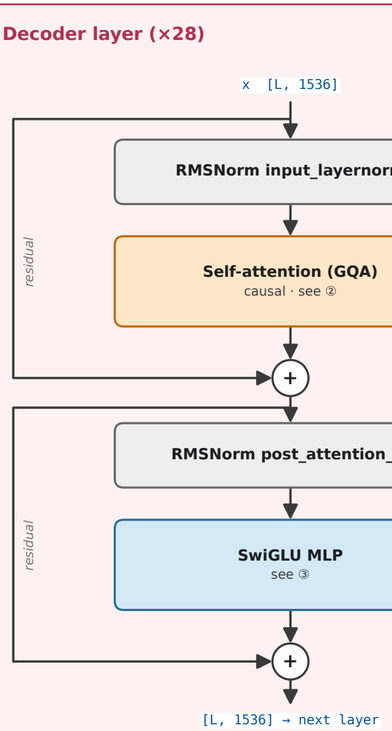
bf16 weights: 6.17 GB · KV cache: 36 KiB per token · ≈ 6.2 GFLOPs per token (linear layers)

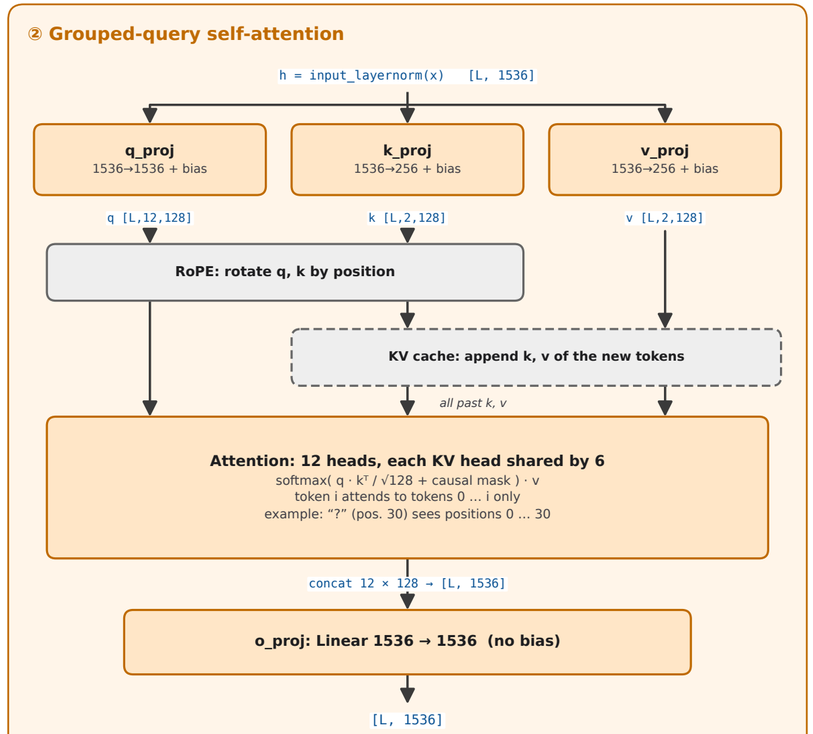
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2.py · Qwen2.5 tokenizer · generated by make_diagrams.py

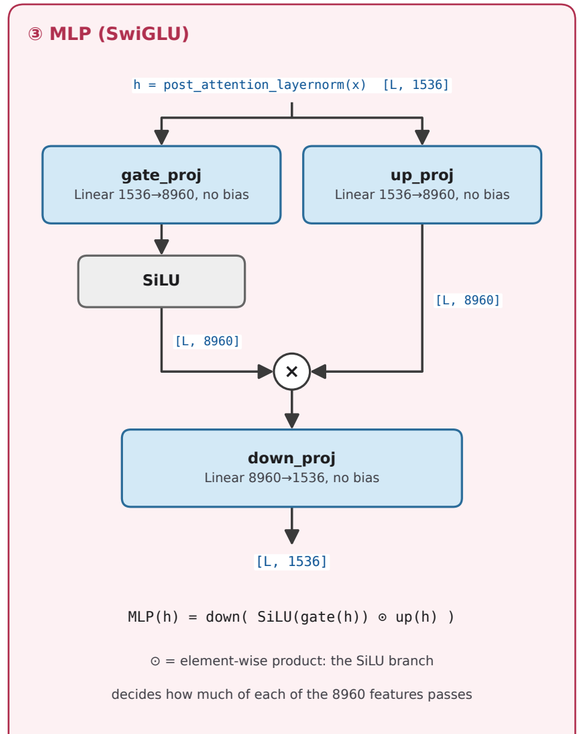
Qwen2.5-1.5B-Instruct — decoder in detail

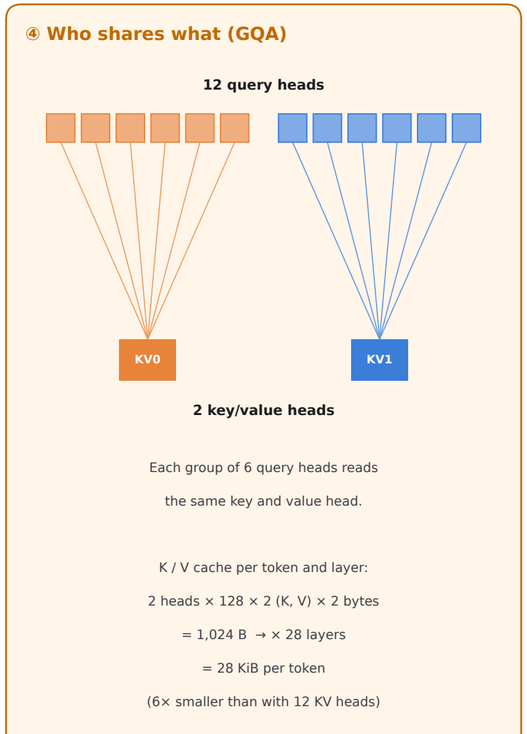
Qwen2 decoder: width 1536, 28 layers, 12 query heads / 2 KV heads × 128, vocab rows 151,936, embedding tied with the LM head. Shapes for the example: L = 36.







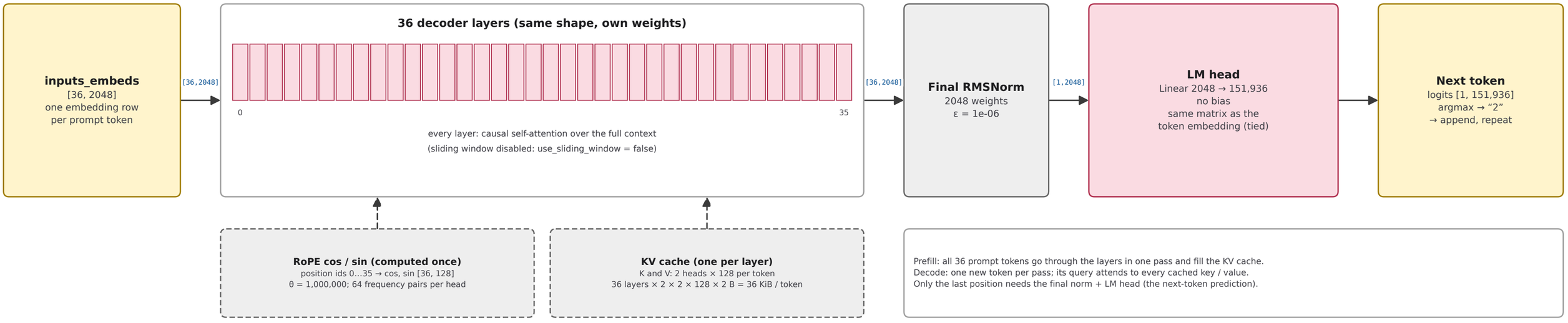


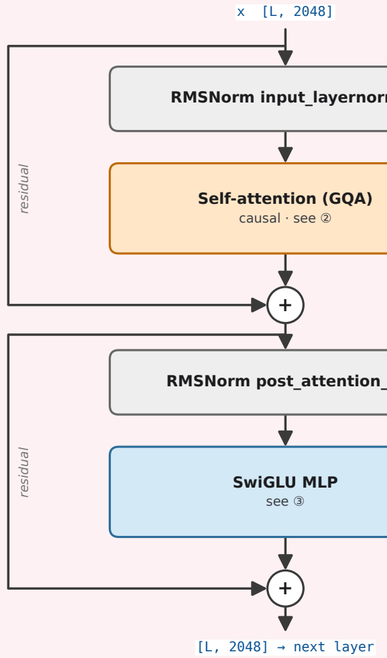


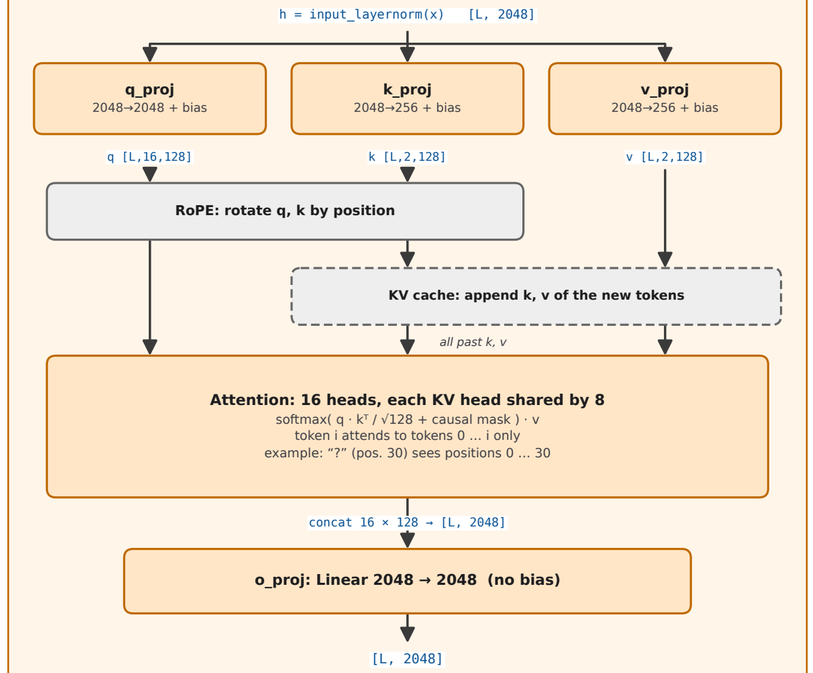
Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2.py · Qwen2.5 tokenizer · generated by make_diagrams.py

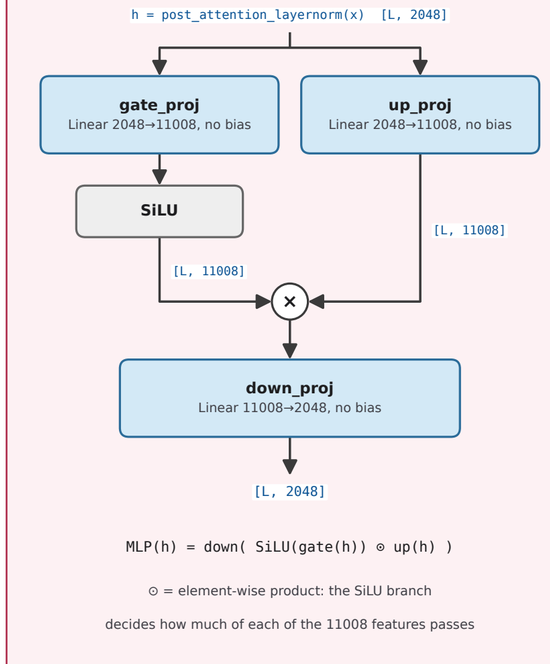
Qwen2.5-3B-Instruct — decoder in detail

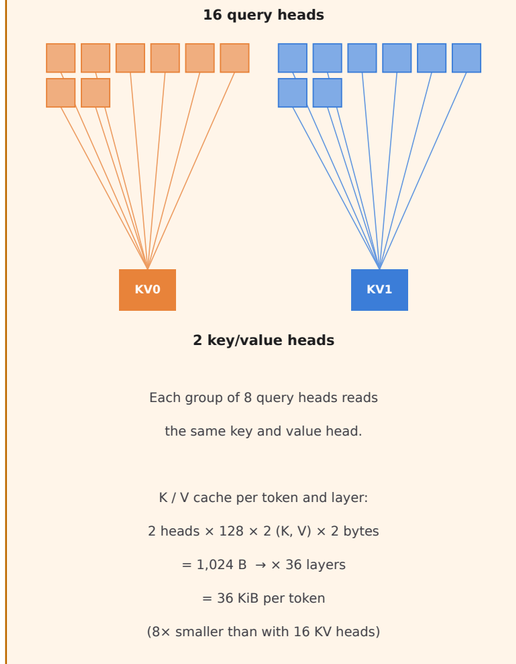
Qwen2 decoder: width 2048, 36 layers, 16 query heads / 2 KV heads × 128, vocab rows 151,936, embedding tied with the LM head. Shapes for the example: L = 36.











Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2.py · Qwen2.5 tokenizer · generated by make_diagrams.py

Prefill and decode — why decoding is limited by memory bandwidth

One pass reads every weight once. Prefill uses each weight for all L prompt tokens; a decode step uses it for one token. Thor's memory: 273 GB/s.

The example on Qwen2.5-3B-Instruct (16-bit weights)

	input	compute	memory read	intensity
Prefill (1 pass)	36 tokens in	36 × 6.2 = 222 GFLOPs	6.17 GB weights	≈ 36 FLOPs / byte
Decode (1 pass each)	1 token in	6.2 GFLOPs	6.17 GB weights + KV	≈ 1 FLOP / byte

Prefill does a lot of work per byte read, so it is limited by compute (matrix × matrix).

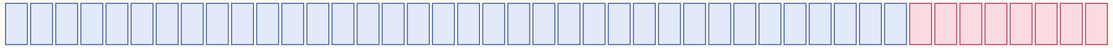
A decode step reads the same weights to produce one token (matrix × vector): it is limited by how fast the weights stream from memory. A lower bound per token:

```
decode ms/token  ≥  bytes read per step / memory bandwidth
```

bytes per step = all layer weights + the whole LM head (V × d) + the KV cache so far; the embedding lookup reads only one row. Smaller weights (8-bit, 4-bit) mean fewer bytes per step, which is why quantization speeds up decoding more than prefill.

	1.5B	3B
bytes per decode step (16-bit)	3.09 GB	6.17 GB
floor at 273 GB/s	11.3 ms	22.6 ms
LM head share of those bytes	15 %	10 %
KV cache at 4,096 tokens	112 MiB	144 MiB

The KV cache grows by one row per token, in every layer



positions 0 … 35: written by prefill — 36 … 43: one per decode step

After the example the cache holds 44 positions, in every layer:

```
3B: 44 × 36 KiB = 1.55 MiB     1.5B: 44 × 28 KiB = 1.20 MiB
```

Each decode step reads the whole cache once (its query attends to every cached key and value). With GQA the cache is small next to the weights:

```
3B, 1,000 tokens: 0.037 GB vs 6.17 GB of weights
```

No reuse across questions: the benchmark turns prefix caching off, so every question starts with an empty cache and a full prefill.

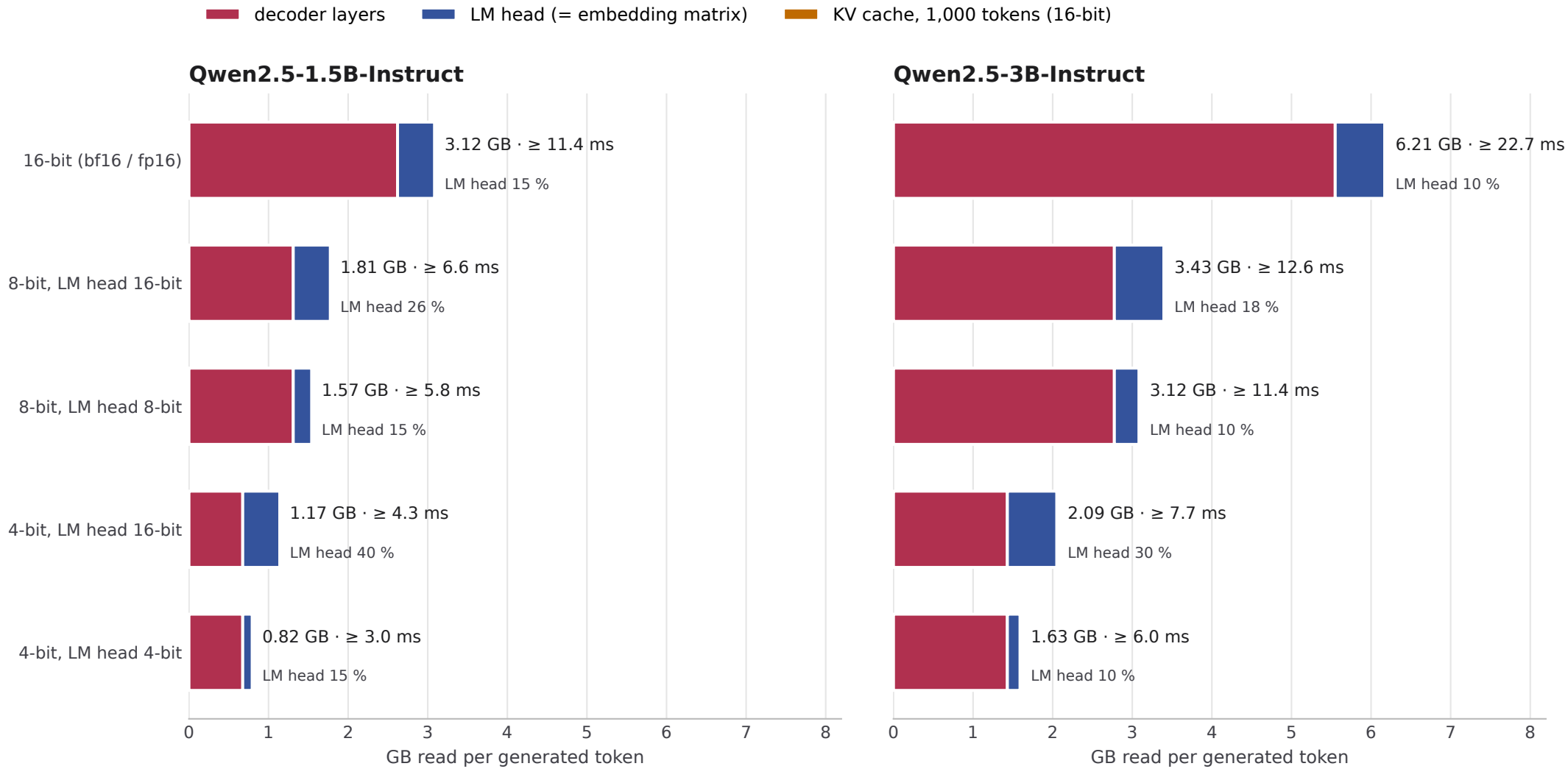
Engines reserve the cache up front for the longest request:

```
context 4,096 → 3B 144 MiB, 1.5B 112 MiB (16-bit; half with fp8 KV)
```

Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2.py · Qwen2.5 tokenizer · generated by make_diagrams.py

Bytes read per decode step, by weight format

Theoretical: layer weights + whole LM head + KV cache at 1,000 tokens. Label = lower bound on decode ms/token at 273 GB/s. 4-bit includes one 16-bit scale per 128 weights.



LM head % = its share of the bytes per step. Real kernels reach only part of the bandwidth, so measured decode times are higher; see jetson/results/thor/FINDINGS_qwen2_5.md. generated by make_diagrams.py

Qwen2.5-1.5B vs 3B — what changes

Same recipe; 3B is wider and deeper. The language model inside Qwen2.5-VL-3B has exactly the 3B shape. Highlighted rows differ between 1.5B and 3B.

	1.5B	3B	VL-3B LLM	note
Parameters	**1.54 B**	**3.09 B**	3.09 B	VL-3B: language model only (+ 0.67 B vision)
bf16 weights	**3.09 GB**	**6.17 GB**	6.17 GB	2 bytes per parameter
hidden size d	**1536**	**2048**	2048	
decoder layers	**28**	**36**	36	
query heads / KV heads	**12 / 2**	**16 / 2**	16 / 2	query heads per KV head: 6 vs 8
head dim	128	128	128	
MLP intermediate size	**8,960**	**11,008**	11,008	5.83 × d vs 5.38 × d
vocab rows (embedding = LM head)	151,936	151,936	151,936	151,665 real tokens, the rest padding; tied in all three
embedding share of the parameters	**15 %**	**10 %**	10 %	the vocabulary is a big part of a small model
RoPE θ · max positions	1,000,000 · 32,768	1,000,000 · 32,768	1,000,000 · 128,000	VL: M-RoPE (equals 1D RoPE for text)
KV cache per token (16-bit)	**28 KiB**	**36 KiB**	36 KiB	layers × 2 × KV heads × 128 × 2 B
GFLOPs per token	**3.1**	**6.2**	6.2	2 × (layer + LM head weights), w/o attention
bytes per decode step (16-bit)	**3.09 GB**	**6.17 GB**	6.17 GB	floor at 273 GB/s: 11.3 vs 22.6 ms/token
LM head share of those bytes	**15 %**	**10 %**	10 %	matters when the layers are quantized and the head is not

Sources: config.json of the checkpoint · transformers 4.57 modeling_qwen2.py · Qwen2.5 tokenizer · generated by make_diagrams.py